A web crawler method based on iframe supporting asynchronous requests

Fudian Qin
Department of Network Engineering, School of Computer Science
Neusoft Institute Guangdong
Foshan, Guangdong, China
qinfudian@nuit.edu.cn

Abstract

Economic analysis and decision-making are inseparable from economic data, and e-commerce platforms aggregate a vast amount of transaction data, user behavior data, and more. Web crawling technology is the primary technique for obtaining data from e-commerce websites. However, due to the impact of anti-crawling technology and website implementation technology, obstacles exist in the execution of crawlers. This paper comprehensively analyzes the characteristics of web crawling technology, anti-crawling technology, and asynchronous request technology on website pages. It suggests that the iframe tag's ability to automatically request and render web pages can be utilized by embedding the iframe tag within web pages. Then, JavaScript scripting can be employed to manipulate the DOM of the web page, parse the web page within the iframe, and obtain data. Verification through e-commerce platforms demonstrates that this method is indeed feasible, simple, and easy to use.

CCS Concepts

• **Applied computing** → Computers in other domains; Personal computers and PC applications.

Keywords

web crawler, anti-crawling technology, Axios, JavaScript

ACM Reference Format:
Fudian Qin. 2025. A web crawler method based on iframe supporting asynchronous requests. In *2025 6th International Conference on Computer Information and Big Data Applications (CIBDA 2025), March 14–16, 2025, Wuhan, China.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3746709.3746831

1 Introduction

In today's information-based society, economic data has become an increasingly diversified source for measuring economic conditions and predicting economic trends. As a bridge connecting consumers and businesses, e-commerce platforms have aggregated vast amounts of transaction data, user behavior data, and more, making them one of the significant sources of economic data. These data not only reflect microeconomic phenomena such as market supply and demand relationships and commodity price fluctuations but also contain macroeconomic information such as consumer preferences and market trends. They play an irreplaceable role in economic analysis and decision-making [1].

However, obtaining economic data is not an easy task. Traditional data collection methods are often time-consuming and laborious, and it is difficult to ensure the comprehensiveness and timeliness of the data. Therefore, web crawling technology emerged as a primary means for acquiring economic data from e-commerce platforms. For example, Vignesh Srinivasan and Chandrasekaran K used a crawler to obtain data on topics related to Musk and Tesla on Twitter and analyze the relationship between this data and their corporate stock prices, which is significant [2]. And Md Khalid Hossen et al. used a crawler to crawl video data from websites and analyzed it statistically to get the relationship between video features and viewers' preferences, which has important implications for market decision-making [3]. By simulating browser behavior, web crawlers automatically capture data from web pages, featuring high efficiency and convenience [4] . However, with the widespread use of crawling technology, issues such as data breaches and increased server pressure on e-commerce platforms have received increasing attention. Consequently, anti-crawling technology has emerged, posing numerous obstacles to the execution of crawlers [5]. At the same time, as web implementation technologies continue to evolve and upgrade, e-commerce platforms are constantly introducing new and innovative technologies. For instance, the asynchronous transmission of data through Ajax technology has further increased the difficulty of data extraction for crawlers [6]. Zheng WenPin argues that asynchronous requests to achieve localized rendering of web pages can not only reduce user waiting time but also prevent crawlers from obtaining all data on the webpage, which is of great significance to website security [7].

In the face of the technology of asynchronous request rendering on webpages, this paper argues that the premise of repeatedly crawling a website is that it should not affect the browsing and reading experience of normal website users. Based on the JavaScript scripting language supported by browsers and the Document Object Model (DOM) of webpages, it is still possible to crawl webpage data. Therefore, this paper analyzes the characteristics of web crawling technology, anti-crawling technology, and asynchronous request technology on website pages, and designs a crawling algorithm to handle asynchronous requests on webpages. This algorithm is then verified through an electric power grid platform.

This work is licensed under a Creative Commons Attribution International 4.0 License.

CIBDA 2025, Wuhan, China
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1316-3/2025/03
https://doi.org/10.1145/3746709.3746831

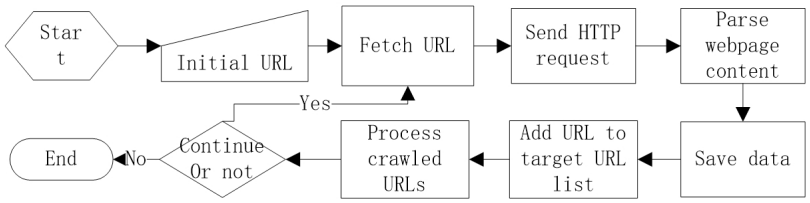

Figure 1: The workflow of a traditional crawler

Table 1: Anti-crawler and crawler programs

Anti-crawler programs	Crawler programs
Determine if the request headers are consistent	Frequently modify the request header
Counting the client visits	Suspend request thread or multi-IP system
Interference the parsing of web content	Artificial intelligence recognizes web content
Enhanced human-computer interaction	Artificial intelligence

2 Web Crawling Technology

A Web Crawler, also known simply as a Crawler or as a Web Spider or Web Robot, is a program designed to collect information from the internet according to specific rules. The basic technologies of a Web Crawler encompass HTTP requests and responses, webpage parsing, and data storage. It serves as a crucial means for automated data collection, with its core principle being the simulation of browser behavior to automatically access webpages and scrape data from them.

The main steps in the workflow of a traditional crawler (Figure 1) are cyclically requesting web page, parsing the response content of the request, saving the data and updating the urls container [8]. The specific algorithm is as follows:

1) Define a urls container and add the target initialization URL.

2) Get a URL from the urls container and doing a web request.

3) Parse the response content of the request and extracting the data.

4) Save the crawled data to a database or file and add the URLs of the web pages to be crawled deeper to the urls container.

It can be seen that the algorithm considers the response content of a URL request containing the data to be crawled, and is only applicable to crawl the data of the traditional websites, and cannot be used for the websites where the data on the web page is requested asynchronously and the data is rendered dynamically.

3 Crawler and Anti-Crawler

Information plays an important role in decision making. Crawlers, as an important channel for getting information, are widely used and continuous development. Along with it, many information security and website crisis problems have arisen, such as personal privacy, website service pressure increased dramatically, and so on. In this regard, the state is improving relevant laws and regulations to protect information security, and enterprises are also improving website technology to cope with the harm of crawlers [9]. Due to the needs of social development, both crawler and anti-crawler technologies are constantly developing, forming a basic balance (Table 1).

The basis for a website to determine whether a client is a crawler is mainly based on the human-computer interaction characteristics of a natural person visiting a website, which are different from those of a crawler. If the client is found to be a crawler, the site restricts, prohibits, or interferes with the crawler's work based on how the crawler works. A natural person's way of accessing a website is to use a computer to view the various pages of the website through a single browser, where switching between pages does not happen quickly, and where each request from the browser to the web server carries the same request header information. A crawler is a program that describes the business logic of the process of access to a website by a natural person or a browser, which is executed automatically by computing and interacts with the web server very quickly. Therefore, it is easy to determine whether a client is a crawler or not based on the difference in access characteristics between a natural person and a crawler. For example, if the access from the same ip address or the same request header is too frequent and faster than normal human operation, these requests are considered as crawler behavior. In this case, the website will limit or prohibit their access, or enhance human-computer interaction by requiring the input of a verification code. For example, Kien Pham, Aécio Santos, and Juliana Freire's research shows that websites respond differently depending on the access characteristics of the user agent and the IP address, and argues that this is a technique for websites to counteract crawlers [10]. Some websites make it more difficult for crawlers to extract webpage data by misplacing webpage data to discourage crawler‘s visiting. This processing method is through CSS or JavaScript technology for misalignment processing, so that the browser rendered data displayed with the response content data of the request is not a simple correspondence, bring about the correct data extracted by the crawler parsing of web pages through the HTML tag structure. For crawler judgment methods with the same ip address or request header, the crawler responds by modifying the request header or the scheme of the multi-ip system to disguise the request as a request from a different user. For example, Katti, a web analytics tool introduced by Florian Nettersheim et al, performs distributed

tasks and works efficiently and reliably [11]. For the accessing frequency statistics by anti-crawler, the crawler responds by using thread blocking to reduce the frequency of requests. For anti-crawler solutions that enhance the difficulty of parsing web pages or increase the behavior of human-computer interaction, the crawler's approach is to train an image recognition model through machine learning techniques and then apply it to the crawler program.

As can be seen from the respective solutions of crawlers and anti-crawlers, they are both based on the basic workflow characteristics of traditional crawlers, i.e., the response content of the web page to a particular URL request contains the data to be crawled. For websites with asynchronous requests in their web pages, the required data may not be extracted, whether parsed by machine learning or by HTML tag structure.

4 Application of Asynchronous Requests in Web Design

Many websites are currently designed with a front-end and back-end architecture built using Vue.js combined with Spring Boot [12, 13]. The back-end project handles the business logic and provides access interfaces for the front-end project to access, and the front-end project is responsible for human-computer interaction and data interaction with the back-end project through Axios. Vue.js has MVVM functionality, which means that it can modify the data on the view when the model data is changed, and it can also change the model data when the data on the view is modified. Considering that the network request is a time-consuming process, the front-end will respond to user requests in a timely manner with web page content to provide user interaction experience, and at the same time, it will use the asynchronous request method of Axios within the hook function of the Vue.js component to query the data from the back-end project and set it to the data model of the Vue.js component, and the model data will be rendered to the view by the Vue.js implementation.

From the current web architecture of front-end and back-end design, the front-end project's response to user requests and asynchronous request for data to the back-end project are really running in two different threads. Traditional crawlers parse and extract data based on the response content of a particular URL request, based on the fact that the response content contains the data to be crawled. As a result, traditional crawlers are no longer suitable for crawling data that is requested asynchronously and rendered dynamically to web pages.

5 Crawler Method of SupportING for asynchronous request

Regardless of how the developed technologies of the web page change, or how advanced the site's anti-crawler technology, the content of the web page ultimately need to be presented to the user through the browser, and the content of the web server's response to the request must conform to the HTML, CSS, JavaScript grammatical standard and the browser's rendering rules. Therefore, crawlers spidering data that is requested asynchronously and dynamically rendered on a web page can be improved by breaking away from the principles of web page composition and the rendering of web browser.

5.1 The composition of the web page and the rendering principles of web browser

When opening and analyzing the source code of a web page, it is easy to see that a web page is a tree-structured document consisting of HTML, CSS, and JavaScript that describes what elements make up the web page, how the page is presented, and what functions it has. HTML tags are the constituent elements of a web page, CSS is responsible for modifying the HTML tag features, and JavaScript is responsible for describing the functional logic of a web page. For example, a window includes glass, frame, handle and other elemental objects, similar to the web page contains a variety of HTML tags, each elemental object of the window has a color, size, position and other characteristics, similar to the CSS description of the characteristics of HTML tags, the window has the function of opening and closing in the left and right directions or front and back directions, similar to the JavaScript description of the function of the web page HTML tags of the logic. The iframe tag is one of the HTML tags, a window embedded in a web page and have the functions of a browser window, such as doing network requests and rendering the content of an HTML document.

The rendering of a web page is the result of a source code file being rendered by a browser. Browser rendering of web pages, first of all, the source code of the HTML tags will be parsed into an object model to build the DOM tree of the web page. At the same time, it also constructs a rendering tree with the CSS characterization of each HTML tag. Then, the browser can draw the components of the web page based on the DOM tree and the rendering tree in the browser window. The JavaScript code in a web page describes a certain behavior of the HTML tag or the function in a certain time. When the page is being rendered, the JavaScript code will also be loaded and enter the pending state. For example, if the login button is clicked by the user, the business function code described by JavaScript such as validate the form data and request to the server will be executed. Another example is that after the body tag is loaded, the function bound to the onload attribute is triggered to execute, and the business logic of the function described by JavaScript is executed. Some of the data on the web page is requested asynchronously and rendered dynamically by implementing it in functions bound to onload attribute.

Programs written in the JavaScript scripting language can be executed directly by the browser, whether they are JavaScript programs in the source code of a web page, or JavaScript code written in the browser's developer tools. Therefore, programmers can write JavaScript code to interactively edit HTML tags in web pages and implement various functions through the browser's developer tools.

Comprehensive analysis of the above techniques, for spidering the data of web pages through asynchronous request and dynamically rendered, can be realized through write and execute the JavaScript crawler code in the browser developer tool. The crawler initiates the request task through the iframe tag. When the iframe tag has finished rendering the response content of the request, it will trigger the execution of the function bound to the onload attribute, thus triggering the asynchronous request and dynamic rendering of data. After that, the crawler parses the page and extracts the data.

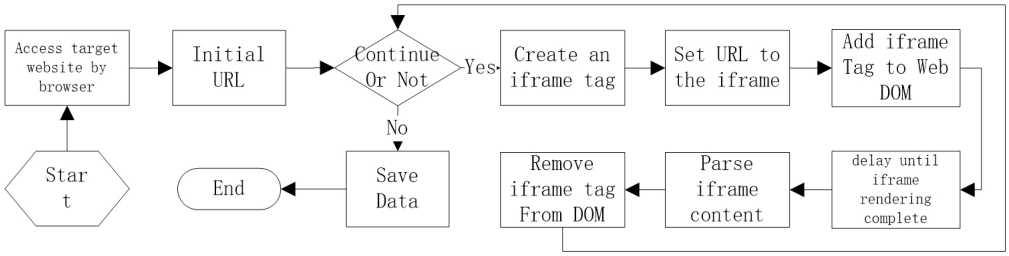

Figure 2: Workflow of a Web Crawler to Address Asynchronous Requests on Webpages

5.2 Workflow of the Crawler Based on iframe to Address Asynchronous Requests on Webpages

See Figure 2, The workflow begins with accessing the target website using a browser. Subsequently, the browser's developer tools code snippet editing window is opened (Source - Snippets - Add Snippet). The crawler program is then written and executed by clicking the triangle execution button below the code snippet (or using the shortcut ctrl+enter). The steps of the crawler program are as follows:

1) Initialize the URL to be crawled. URLs for deep crawling can be added during subsequent webpage parsing.

2) Create an iframe tag element.

3) Obtain the URL and set it in the src attribute of the iframe tag, while also configuring other attributes such as id.

4) Append the iframe tag element to a specific node in the webpage's DOM, such as under a form or body tag. The browser will then parse and render the iframe element, initiating a request in a new window. If there are asynchronous requests for dynamic rendering during the webpage rendering in the new window, they will be automatically executed.

5) Execute a delay function to wait for the iframe window to render completely.

6) Parse the content of the iframe window, extract data, save the data, and process URLs, for example, by adding them to the list of URLs to be crawled.

7) Remove the iframe window through its parent tag element object.

8) Repeat the above steps. Once all URLs have been crawled, save all collected data to a file and terminate the crawler program.

6 Use Case of Web Crawlers

Taking the task of crawling product information from Taobao.com as an example, the product data on Taobao's product listing pages are rendered dynamically through asynchronous requests. If traditional crawler technology is used, the response content will not contain the product data. However, using the iframe request rendering method can effectively solve this problem. Firstly, open a browser and login to Taobao (https://uland.taobao.com/sem/tbsearch?clk1$=$f75e6af005899b266b576d16b947f426). Then, write the crawler program in the code snippet editing window. Finally, execute the crawler program by clicking the triangle execution button below the code snippet (or using the shortcut ctrl+enter). The crawler program is shown in Table 2.

7 Conclusions

This web crawler method is not only easy to use, but also get the data on the web page through asynchronous request and dynamic rendering. The reasons for easy to use are those the programming language is JavaScript and the program execution environment is the browser. And the ability to get the data through asynchronous request and dynamic rendering on the web page is due to the inline frame characteristic of the iframe tag. The onload function can be executed after the content document of the iframe tag is loaded. So in the onload function it is possible to request data asynchronously and render it to the web page. This process is equivalent to a web crawler making two requests during the request process before parsing the page content.

This method contains weaknesses in two aspects. On the one hand, it is not easy to use the data. Because the data is saved to a file not in a database. On the other hand, the time to parsing web page can't be sure. Because the web page parsing must has been rendered and the time token to rendering is not the same for different pages.

Acknowledgments

This article was supported by Teaching Quality and Teaching Reform Project of Guangdong Undergraduate Colleges and Universities in 2024: 'Teaching Construction and Reform of Mobile Application Development Course Based on HarmonyOS in the Context of New Engineering' (No. 30, Guangdong Higher Education Letter [2024]), Young Innovative Talent Category Project of Guangdong Ordinary Colleges and Universities in 2023: 'Research on MIMO radar near-field target parameter estimation method in complex scenarios'(2023KQNCX111), Characteristic Innovative Category Project of Guangdong Ordinary Colleges and Universities in 2023: 'Research and applying of key technologies in optimizing the parameters of industrial equipment adapted to smart manufacturing'(2023KTSCX194) and Visiting Scholar Research Project in 2024: 'Research and Development of Economic Data Analysis and Mining Software System'.

References

[1] Yu Bo, Feng Wenwen, Yu Xiaoyu, *et al.* Data Collection and Sentiment Analysis of New Energy Vehicle Reviews Based on Scrapy [J]. Computer Knowledge and Technology, 2024, 20(19): 33-35.

Table 2: Example of a Web Crawler Program

Steps	Code

Initialize request URL	var urls=[];// URL of the product listing page
	var detailUrls=[];// This URL is used when crawling product details
	//Analyze the page and store the URLs of all pages in the pagination into the urls array.
	//For example
	for(var i=1;i<100;i++){
	var url =
	'https://uland.taobao.com/sem/tbsearch?clk1=f75e6af005899b266b576d16b947f426
	&localImgKey=&page='+i+'&q=u%E7%9B%98&tab=all';
	urls.push(url);
	}

Define data container	var mydb = new Array();

Delay function	function delay(ms) {
	return new Promise(resolve => setTimeout(resolve, ms));
	}

Loop request and parse web page getting data	async function myCrawler() {
	while(urls.isNotEmpty()){
	// Adding iframe tag to outer web page and set the url of the crawling page to the src //attribute of the adding iframe tag.
	var divtag= document.querySelector("#useForScrolltop");
	var iframetag = document.createElement('iframe');
	iframetag.src= urls.pop() ;
	iframetag.id='subiframe';
	divtag.appendChild(iframetag);
	// Delay 200 milliseconds
	await delay(200);
	// Parse web page
	var embeddediframe = document.getElementById("subiframe");
	var embeddedwindow = embeddediframe.contentWindow;
	var contentToBeParsed = embeddedwindow.document;
	//Define array for saving parsed datas
	var parseddata = [];
	var dataItems =
	contentToBeParsed.querySelectorAll('[class^="Card−doubleCardWrapper"]');
	for(index in dataItems){
	var dataObject={};// Define an object for saving one record of parsed datas
	var leafNodeTage= dataItems [index]['parentElement']['lastChild'];
	// push the url of detailed page to be crawled into detailUrls
	detailUrls.push(dataItems [index]['href']);
	picElementTage= leafNodeTage.querySelector('[class^="MainPic−mainPic"]');
	dataObject['mainPic']=picElementTag.src;
	// Continue to get other data items for the record
	parseddata.push(dataObject);
	}
	mydb=mydb.concat(parseddata);
	embeddediframe.parentElement.removeChild(embeddediframe);
	}
	}
	myCrawler ();

Save data	//Adding JavaScript code library
	var scripttag=document.createElement('script');
	scripttag.src="https://cdnjs.cloudflare.com/ajax/libs/FileSaver.js/2.0.5/FileSaver.min.js";
	var headtag=document.querySelector('head');
	headtag.appendChild(scripttag);
	//Saving the data of mydb variable
	var blobdata = new Blob(mydb, {type: "text/plain;charset=utf-8"});
	saveAs(blobdata,"mydata.json");

[2] Vignesh Srinivasan and Chandrasekaran K. 2021. Data Aggregation Of Tweets And Topic Modelling Based On The Twitter Dataset. In Proceedings of the 2021 3rd International Conference on Big Data Engineering and Technology (BDET '21). Association for Computing Machinery, New York, NY, USA, 15–20. https://doi.org/10.1145/3474944.3474947

[3] Md Khalid Hossen, Yong Wang, Hussain Ahmed Tariq, Gabriel Nyame, and Raphael Elimeli Nuhoho. 2018. Statistical Analysis of Extracted Data from Video Site by Using Web Crawler. In Proceedings of the 2018 International Conference on Computing and Artificial Intelligence (ICCAI '18). Association for Computing Machinery, New York, NY, USA, 41–46. https://doi.org/10.1145/3194452.3194466

[4] Meng Xianying, Mao Yingshuang. Commodity Information Collection and Analysis Based on Python Crawler Technology [J]. Software, 2021, 42(11): 128-130.

[5] Wang Shuai. Design and Implementation of Anti-crawler Programs Based on the Selenium Framework [J]. Information Recording Materials, 2023, 24(06): 86-88.

[6] Guo Jinyu. Design and Implementation of an Anti-crawler System Based on Spark Streaming [D]. Xidian University, 2021.

[7] Zheng Wenping. Research on Web Crawling and Anti-crawling Technologies [J]. Computer Programming Skills & Maintenance,2022,(12):173-176.

[8] Cui Huanhuan. Research on Web Crawling Technology Based on Python [J]. Information Recording Materials, 2023, 24(06): 172-174.

[9] Cao Wenbin, Zhang Kejing. Research on Anti-crawler Technology Based on Automated Testing: Taking Tmall as an Example [J]. Modern Computers (Professional Edition), 2018, (11): 64-67.

[10] Kien Pham, Aécio Santos, and Juliana Freire. 2016. Understanding Website Behavior based on User Agent. In Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval (SIGIR '16). Association for Computing Machinery, New York, NY, USA, 1053–1056. https://doi.org/10.1145/2911451.2914757

[11] Florian Nettersheim, Stephan Arlt, Michael Rademacher, and Florian Dehling. 2023. Katti: An Extensive and Scalable Tool for Website Analyses. In Companion Proceedings of the ACM Web Conference 2023 (WWW '23 Companion). Association for Computing Machinery, New York, NY, USA, 217–220. https://doi.org/10.1145/3543873.3587351

[12] Jia Wenqiang, Liu Xin, Fu Peng. Design and Implementation of an Enterprise Record Management System Based on the Spring Boot + Vue Framework [J]. Industrial Control Computers, 2024, 37(10): 151-152.

[13] Cao Hao, Li Jie, Xie Bin. Design of a Tourism Information System for Longsheng Autonomous County, Guilin, Based on SpringBoot + Vue [J]. Modern Information Technology, 2024, 8(16): 102-106.